

Software Verification and Validation

Part Time Lecturer
Talimul Bari Shreshtho

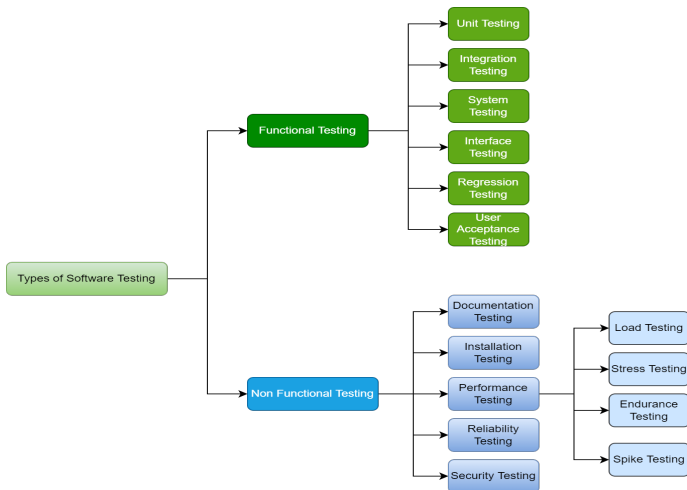
Islamic University of Technology

August 5, 2026



Table of Contents

Types of Software Testing



Integration Testing

Definition

Integration Testing is a testing level in which individually tested software modules are combined and tested as a group to verify that they interact correctly through their interfaces.

Focus of Integration Testing

- Communication between modules
- Data exchange across interfaces
- Function calls and returned values
- Interaction with external components

Key Idea

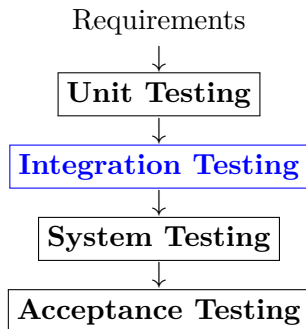
Even if individual modules pass unit testing, defects may still occur when the modules are integrated.

Objective and Position of Integration Testing

Objectives

- Verify correct interaction between integrated modules.
- Detect interface and communication defects.
- Ensure data is exchanged correctly.
- Validate the behavior of interconnected components before system testing.

Testing Levels



Why Integration Testing?

Passing unit tests does **not** guarantee that the complete system will function correctly. When independently tested modules are integrated, new defects may appear due to their interactions.

Integration testing helps detect:

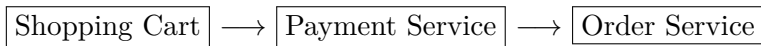
- **Interaction Failures**
 - Modules work correctly in isolation but fail when communicating.
- **Interface Defects**
 - Incorrect parameters, incompatible interfaces, or API mismatches.
- **Data Flow Problems**
 - Incorrect or inconsistent data passed between modules.
- **Communication Issues**
 - Missing function calls, incorrect message exchange, or synchronization failures.

Key Idea

Integration testing focuses on verifying the **connections between modules**, rather than testing the individual modules themselves.

Example: Why Integration Testing?

E-Commerce System



Each module passes **Unit Testing**, but integration may reveal:

- Payment service returns an unexpected response format.
- Order service receives incorrect transaction data.
- Communication timeout between services.
- Incorrect API endpoint or authentication failure.

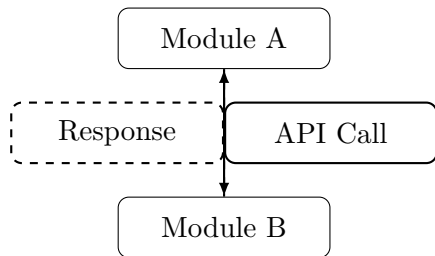
Observation

These defects cannot usually be detected during Unit Testing because they occur only when modules interact.

Common Integration Defects

Integration testing focuses on finding defects that occur when modules communicate with each other.

- Interface mismatch
- Incorrect parameter passing
- Data format mismatch
- Timing and synchronization errors
- Exception propagation failures

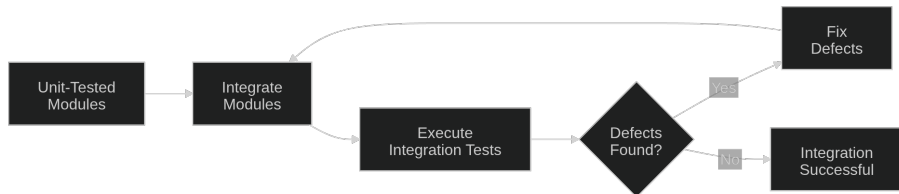


Communication between two software modules.

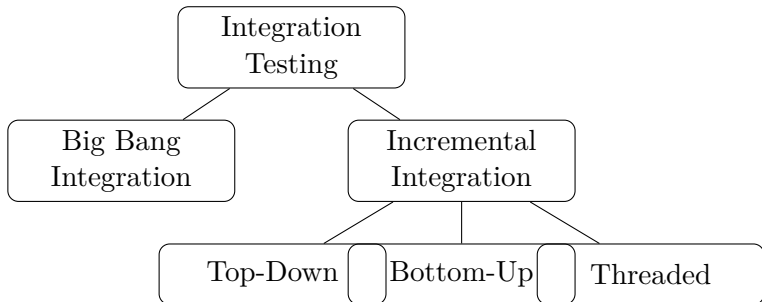
Key Idea

Even if each module passes unit testing, defects may still occur because of incorrect interactions between modules.

Process



Types of Integration Testing



Classification

- **Big Bang Integration:** All modules are integrated at once.
- **Incremental Integration:** Modules are integrated step by step.
 - Top-Down
 - Bottom-Up
 - Threaded

Stubs and Drivers

During integration testing, some modules may not yet be available. To continue testing, **temporary modules** are created to simulate their behavior.

Stub

A **Stub** replaces a lower-level module that has not yet been developed. It returns predefined responses to the calling module.

Driver

A **Driver** replaces a higher-level module that has not yet been developed. It invokes the lower-level module and provides test inputs.

Remember

- **Top-Down Integration** → Uses **Stubs**
- **Bottom-Up Integration** → Uses **Drivers**

Stub Example

Scenario

The LoginService is complete, but the DatabaseService is not.

Login Service

```
boolean login(String user, String pass){  
    return database.verify(user, pass);  
}
```

Stub for Database Service

```
class DatabaseStub{  
    boolean verify(String u, String p){  
        return true; // predefined response  
    }  
}
```

Purpose

The stub simulates the missing lower-level module so that the higher-level module can be tested.

Driver Example

Scenario

The Calculator module is complete, but the Main Program is not.

Calculator Module

```
int add(int a, int b){  
    return a + b;  
}
```

Driver

```
public static void main(String[] args){  
    Calculator c = new Calculator();  
    System.out.println(c.add(3,5));  
}
```

Purpose

The driver simulates the missing higher-level module by calling the lower-level module with test inputs.

Stub vs Driver

Stub	Driver
Simulates lower-level module	Simulates higher-level module
Returns predefined outputs	Provides test inputs
Used in Top-Down Integration	Used in Bottom-Up Integration
Called by the module under test	Calls the module under test

Top-Down → Stub Bottom-Up → Driver
--

Big Bang Integration Testing

Definition

In **Big Bang Integration**, all unit-tested modules are combined simultaneously and tested as a complete system.

Advantages

- Simple integration approach.
- No need for stubs or drivers.
- Suitable for small systems with few modules.

Disadvantages

- Difficult to isolate defects.
- Interface errors are harder to locate.
- Integration starts only after all modules are completed.
- High debugging effort.

Big Bang Integration Testing

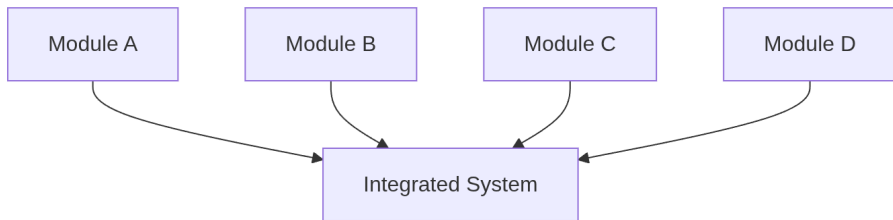


Figure 1: Big Bang

Top-Down Integration Testing

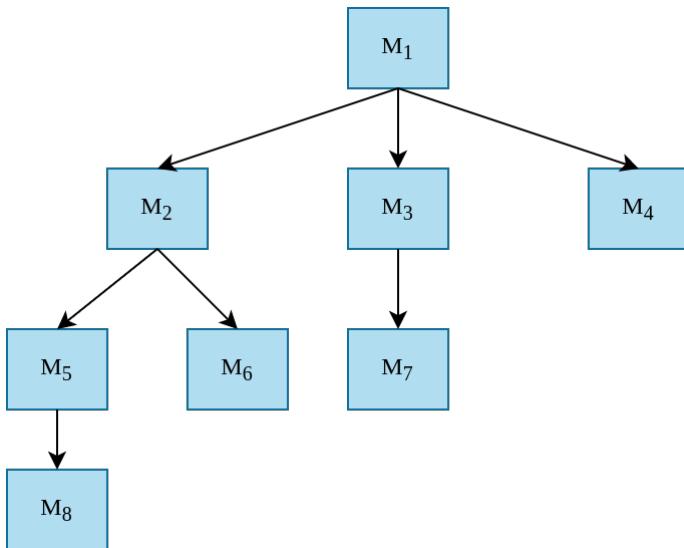
Process

Integration begins with the **top-level module**. Lower-level modules are added one by one until the entire system is integrated.

Uses

- **Stubs** to simulate lower-level modules that are not yet available.

Top-Down Integration Orders



Top-Down Integration Orders

In **Top-Down Integration Testing**, modules are integrated from the highest level to the lowest level following the system architecture.

Two common traversal strategies are used:

Breadth-First Search (BFS)

Modules at the same level are integrated and tested before moving to the next lower level.

$$M1 \rightarrow (M2, M3, M4) \rightarrow (M5, M6, M7) \rightarrow M8$$

Depth-First Search (DFS)

One complete branch is integrated and tested before moving to another branch.

$$M1 \rightarrow M2 \rightarrow M5 \rightarrow M8 \Rightarrow M6 \Rightarrow M3 \rightarrow M7 \Rightarrow M4$$

Top-Down Integration Testing

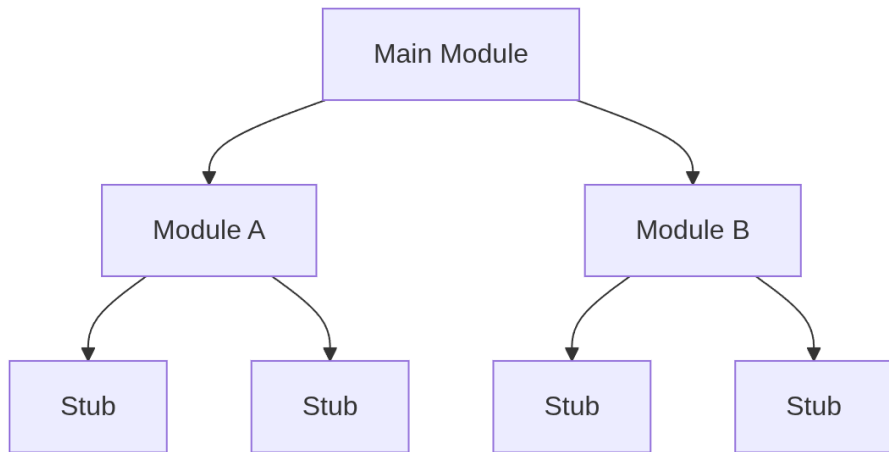


Figure 2: Top Down

Advantages

- Early validation of high-level logic.
- Major design defects are detected early.
- No drivers are required.
- Extremely consistent and less time required
- Less time required

Disadvantages

- Many stubs may be required.
- Low-level modules are tested later.
- Stubs may not accurately represent real module behavior.

Bottom-Up Integration Testing

Process

Integration begins with the **lowest-level modules**. Higher-level modules are added gradually until the complete system is integrated.

Uses

- **Drivers** to simulate higher-level modules that are not yet available.

Advantages

- Low-level utility modules are thoroughly tested first.
- Easier to isolate and debug interface defects.
- No need to develop stubs.
- Parallel development and testing are possible.

Disadvantages

- High-level functionality is tested late.
- Many drivers may be required.
- No early working prototype of the system.

Bottom-Up Integration Testing

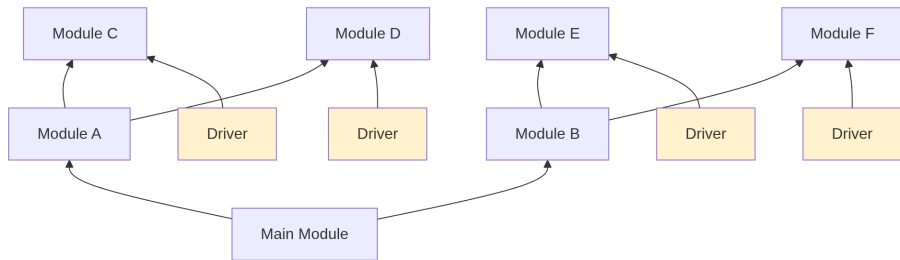


Figure 3: Bottom Up

Sandwich (Hybrid) Integration Testing

Definition

Sandwich Integration (also called **Hybrid Integration**) combines the **Top-Down** and **Bottom-Up** integration strategies. Integration proceeds simultaneously from the top and bottom levels until the middle layer is reached.

Characteristics

- Combines Top-Down and Bottom-Up approaches.
- Uses both **Stubs** and **Drivers**.
- Multiple teams can perform integration in parallel.
- Suitable for large and complex software systems.

Sandwich (Hybrid) Integration Testing

Advantages

- Faster integration process.
- High-level and low-level modules are tested simultaneously.
- Reduces overall testing time.

Disadvantages

- More complex planning and coordination.
- Requires both stubs and drivers.

Sandwich (Hybrid) Integration Testing

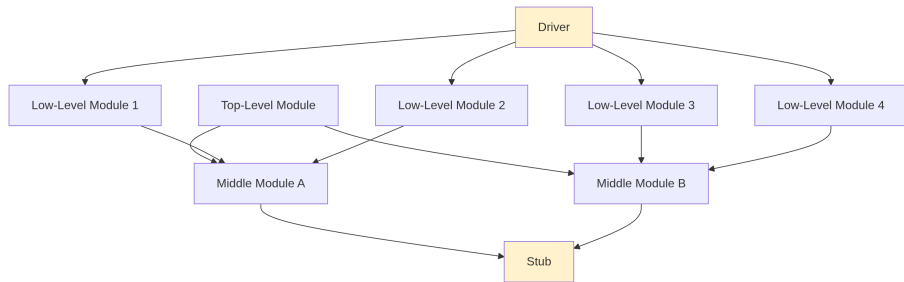


Figure 4: Sandwich Testing

Choosing a Sandwich (Hybrid) Integration Strategy

The effectiveness of the Sandwich (Hybrid) approach depends on several project-specific factors.

Factors to Consider

- **Priority of Subsystems**
 - Integrate critical or high-risk subsystems as early as possible.
- **User Interface Requirements**
 - If early user interface validation is important, begin with **Top-Down** integration.
- **Machine Interfaces and Performance**
 - If low-level hardware interaction or performance is critical, prioritize **Bottom-Up** integration.
- **Availability of Modules**
 - The integration order should consider which modules are already developed and ready for testing.

Key Idea

The Sandwich strategy allows teams to combine Top-Down and Bottom-Up integration based on project priorities, system architecture, and module availability.

Comparison of Integration Strategies

Strategy	Stubs	Drivers	E Prototype	Debugging
Big Bang	No	No	No	Difficult
Top-Down	Yes	No	Yes	Moderate
Bottom-Up	No	Yes	No	Easy
Sandwich	Yes	Yes	Partial	Moderate
Threaded	Sometimes	Sometimes	Yes	Easy

Key Takeaways

- **Big Bang** is simple but makes debugging difficult.
- **Top-Down** validates the system architecture early using **stubs**.
- **Bottom-Up** thoroughly tests lower-level modules using **drivers**.
- **Sandwich** combines the advantages of Top-Down and Bottom-Up.
- **Threaded** integrates one feature (thread) at a time, enabling early functional testing.

Example of Integration Testing

Scenario

Consider an email application consisting of three modules:

- **Login Page**
- **Mailbox**
- **Delete Email**

These modules are individually tested using **Unit Testing** and are then integrated to verify that they work correctly together.



Integration Test Scenarios

- Verify that a valid login opens the correct mailbox.
- Verify that mailbox contents are loaded successfully after login.
- Verify that deleting an email removes it from the mailbox.
- Verify that the mailbox refreshes after deletion.

Example of Integration Testing

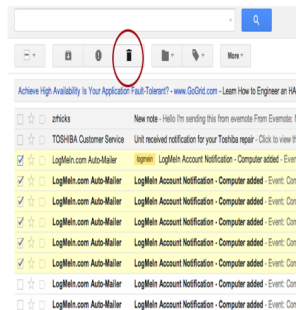
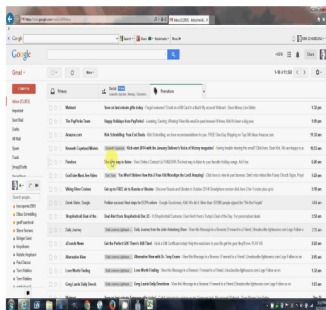


One account. All of Google.

Sign in with your Google Account

A screenshot of the Google sign-in page. It features a large grey circle with a white person icon. Below it is a text input field for 'Email' and another for 'Password'. A blue 'Sign in' button is positioned below the password field. To the left of the button is a checked checkbox labeled 'Stay signed in' and a link 'Need help?'. Below the sign-in area is a link 'Create an account'.

One Google Account for everything Google



Example Integration Test Cases

ID	Objective	Description	Expected Output
IT-01	Verify the interface between the Login Page and Mailbox	Enter valid username and password, then click the Login button.	The user is successfully authenticated and redirected to the Mailbox page.
IT-02	Verify mailbox loading after login	Log in with a valid user account.	The mailbox displays all emails belonging to the logged-in user.
IT-03	Verify the interface between Mailbox and Delete Email	Select an email from the mailbox and click the Delete button.	The selected email is removed from the mailbox and the inbox is updated.
IT-04	Verify mailbox refresh after deletion	Delete an email and refresh the mailbox view.	The deleted email is no longer displayed, and the total email count is updated correctly.

Challenges of Integration Testing

Integration testing ensures that software modules work together correctly, but it also presents several practical challenges.

- **Complex Debugging**

- Failures may involve multiple interacting modules, making root cause analysis difficult.

- **Dependency Issues**

- Some modules, APIs, or external services may not yet be available or stable.

- **Test Environment Setup**

- Requires an environment that closely resembles production.

- **Test Data Management**

- Shared databases and services require consistent and reliable test data.

- **Supporting Components**

- Additional effort is needed to develop stubs, drivers, and mocks.

Tools for Integration Testing

Various tools help automate integration testing, simulate dependencies, and verify interactions between software components.

Tool	Purpose
Postman	API testing and request-response verification
RestAssured	Java library for automated REST API testing
Testcontainers	Run databases and services in Docker containers for realistic testing
JUnit / TestNG	Frameworks for writing and executing integration tests
WireMock	Mock external APIs and third-party services
Pytest	Python framework for writing integration tests
Jenkins / GitHub Actions	Automatically execute integration tests in CI/CD pipelines

Best Practices for Integration Testing

Following best practices improves the reliability and maintainability of integration testing.

- Begin integration testing only after individual modules pass unit testing.
- Prioritize testing of critical interfaces and high-risk module interactions.
- Use realistic, consistent, and well-managed test data.
- Maintain a stable test environment that resembles production.
- Automate repetitive integration tests whenever possible.
- Use mocks, stubs, and drivers to isolate unavailable components.
- Perform regression testing after every defect fix or system update.
- Integrate testing into the CI/CD pipeline to detect issues early.

Thank You

Thank you for your attention!

`talimulbari@iut-dhaka.edu`